HACKATHON: WE CODE CODERS | TEAM CARTOON

INKLOOM AI
Round 3, Improvement 1
Secure Database and Multi-User Access

Detailed master implementation prompt. Paste Parts 1 to 12 into your AI coding agent (AI Studio, Claude Code, Cursor, etc.) to take the project from a working demo to a genuinely persistent, secure, multi-user system.

What this prompt was built from
Your Round 1-3 proposal PDF **and** a line-by-line audit of your uploaded project zip (server.ts, server/auth.ts, server/db/repos.ts, src/api/client.ts, src/services/store.ts, src/services/supabase.ts, db/ migrations, seed and tests). Every task below targets a gap that exists in the code today, not a generic wish list.

Honest headline finding
Your docs describe a Supabase-backed system with RLS, but at runtime the Express repositories are **in-memory JavaScript arrays** (data vanishes on restart), the migration runner **never executes SQL**, and the React client mutates a **browser localStorage store** for almost every action (only login, logout, health and one users call hit the server). This is exactly what your Round 3 slide calls 'partially implemented'. The prompt below closes that gap.

Scope guard: this prompt covers only Improvement 1 (Secure Database and Multi-User Access). Benchmarking/regression testing (Improvement 2) and automated image checking (Improvement 3) are explicitly out of scope.

How to use this document

- **Parts 1-2** are context for the agent. Parts 3-11 are the ordered work plan. Part 12 is the final report format.
- Give the agent your project zip/repo **plus** this PDF. Tell it: *'Follow the INKLOOM AI Round 3 Improvement 1 master prompt exactly, one phase at a time, and stop at each checkpoint to show command output.'*
- Run it phase by phase. Do not let the agent do all phases in one giant response; the client rewrite (Phase 7) is large and needs its own session.
- Before Phase 2 you must create (or confirm) a Supabase project and provide **DATABASE_URL**, **SUPABASE_URL** and **SUPABASE_SERVICE_ROLE_KEY** as environment secrets. Use a **separate test project** (or schema) for automated tests.
- Rotate the Supabase anon key that is currently hard-coded in `src/services/supabase.ts`, and treat the project URL in README/docs as public but unneeded.

Audit snapshot: what the zip actually does today

#	Finding (evidence)	Why it matters	Sev
1	`server/db/repos.ts` holds all data in module-level arrays (`let users = [...SEED_USERS]`, `submissions = []`, etc.). Supabase is only used by `checkHealth()`.	Restart or redeploy wipes submissions, approvals, comments and new users. Not a database.	Critical
2	`src/api/client.ts`: only 4 of ~47 methods call `fetch`. The rest read/write `store` (in-browser) and call `store.persist()` to `localStorage['inkloom_store_v1']`. The AI analysis runs in the browser.	All server RBAC, self-approval guard, validation and audit logging can be bypassed by the UI path. Each browser has its own private world; no real multi-user collaboration.	Critical
3	`db/migrate.ts` lists files then prints 'All migrations verified'. It never runs the SQL. `db/seed.ts` upserts inside try/catch and still prints 'Seeding completed successfully'.	Schema may never have been applied; failures are silent and reported as success.	High
4	Schema uses `uuid` primary keys, but seed IDs are strings like `'org-lumora-001'`, `'u-maya-admin'`; `server.ts` mints `` `org-${Date.now()}` ``. Social tables use text IDs. `001_init.sql` and `schema.sql` duplicate each other.	Real inserts into Postgres will fail on type errors. Two sources of truth for the schema.	High
5	`003_rls.sql` enables RLS on 17 tables but defines policies for only 4, using an `app.current_org` setting that nothing ever sets. Server would use a key that bypasses RLS anyway. 'Org' in docs = `org_id` uuid cast.	RLS is decorative. Tenant isolation currently relies on repo code only.	High
6	`PATCH /api/suggestions/:id` calls `suggestionRepo.getById(id)` with no org/role/ownership check; `reportRepo.getByVersionId` is also unscoped.	Cross-organisation IDOR: a user in org A can modify a suggestion belonging to org B if the ID is known.	High
7	`server/auth.ts` falls back to `'inkloom-ai-default-jwt-secret-key-32charsmin'`; the fail-fast check exists only when NODE_ENV=production. `failed_login_count` exists in the schema but `server.ts` never uses it (no lockout). No logout revocation, no refresh tokens.	Forgeable tokens if the env var is missing; no brute-force lockout; stolen JWT is valid for 2 hours even after logout.	High
8	`src/services/supabase.ts` is imported by both server and browser bundle and hard-codes a project URL and anon key as fallbacks. Gemini key is stored in `localStorage` (AiEngineModal, SettingsPage) and passed into the pipeline from the client.	Violates the proposal's own trust boundary ('browser never talks to DB/LLM; keys only on server').	High

#	Finding (evidence)	Why it matters	Sev
9	Social routes `GET /api/social/accounts` and `/providers` have no `requireAuth`; `POST /api/social/*` has auth but no role check; comments POST allows any authenticated role including viewer; `CORS origin` falls back to `true`.	RBAC incomplete: your own matrix says viewers are read-only.	Medium
10	`db/seed.ts` has admin, creators, reviewers but the README promises a **viewer** persona (Kavita Iyer). The only seed user list that includes her may not exist in the DB path.	Viewer role cannot be demonstrated or tested end-to-end.	Medium

PART 1 - Role, objective and non-negotiable rules

1.1 Role you (the coding agent) play

You are a senior full-stack engineer and security reviewer finishing **Round 3, Improvement 1** of INKLOOM AI, a reference-grounded brand governance platform (React 19 + Vite + Tailwind client; Express + TypeScript server; Supabase PostgreSQL; Gemini behind a server-side pipeline; Vitest tests). You are completing an existing implementation, not rebuilding it. Preserve every existing feature, route path, response shape and governance guard unless this prompt explicitly says to change it.

1.2 Objective (from the team's Round 3 plan)

Integrate persistent database storage, secure authentication and role-based access control. Brand profiles, guidelines, submissions, corrections and review history are stored reliably and survive sessions and restarts. Teams collaborate in a shared workspace, return to earlier submissions and keep their work instead of relying on temporary or browser-local storage. Each user accesses only what their role permits. Complete the database integration, implement secure sign-in and session handling, and enforce permissions on the backend as well as in the interface.

1.3 Definition of success in one paragraph

Two people on two different browsers (e.g. a creator and a reviewer) can log in, work on the same organisation's data, stop and restart the server, and find everything still there. Every API call is authenticated, role-checked and organisation-scoped on the server; the browser holds no business data, no secrets and no AI keys; an attacker with a valid account in another organisation, a viewer account, or a stolen/expired/revoked token cannot read or change what they should not. All of this is proven by automated tests that run against a real Postgres database.

1.4 Non-negotiable rules

- **Server is the only authority.** The browser never talks to Supabase, Postgres, or Gemini. Remove all Supabase client usage from `src/` (bundle must contain no Supabase URL, anon key or service-role key).
- **Secrets live only in server environment variables.** No default or fallback value for JWT_SECRET, SUPABASE keys or DATABASE_URL in any environment except an explicit, clearly named test setup. The server must refuse to boot if a required secret is missing or weak.
- **Every query is organisation-scoped** at the repository layer *and* backed by database-level defence in depth (RLS deny-by-default for the public roles, plus org_id on every tenant table).
- **No silent success.** Migration, seed and sync scripts must exit non-zero and print the real error when any step fails. Never log 'completed successfully' after a caught failure.
- **Humans decide.** Do not weaken any existing governance guard: AI never approves; self-approval is blocked; unresolved high findings block send-for-review; last-admin protection; append-only audit logs, approval records and versions.
- **Do not fake it.** If something cannot be completed (no database credentials, a test cannot run), say so in the final report. Do not stub with in-memory data and call it persistent. No fabricated test output.
- **Keep public contracts stable.** Existing route paths, JSON envelopes (`{ data }` / `{ error, message }`) and TypeScript types in `src/types/inkloom.ts` stay compatible. Extend, do not rename. Update `docs/API_CONTRACT.md` and keep `npm run contract:check` green.
- **Small, reviewable steps.** One phase per commit-sized change. After each phase run `npm run lint`, `npm test` and `npm run build` and paste results.

PART 2 - Target architecture

```
Browser (React)                 Express API (only trusted tier)             Supabase Postgres
-------------------             -------------------------------            ----------------------
No DB client                    helmet, CORS allow-list, rate limits        Tenant tables, all with
No keys, no seed data   --HTTPS-> requireAuth (access JWT, 15 min)          org_id + FK + indexes
Access token in memory          requirePermission(<permission>)             RLS ON, deny-by-default
Refresh token: httpOnly         org-scoped repositories (async, pg pool)    auth_sessions (hashed
cookie (SameSite=Strict)        zod validation on every route               refresh tokens, revoked_at)
Server state via api client     status machine + governance guards          audit_logs (append-only)
                                AI pipeline runs HERE (Gemini key here)     schema_migrations table
```

Key decisions the agent must follow (do not re-litigate)

Decision	Choice	Reason
DB access from server	`pg` Pool via **DATABASE_URL** (Supabase pooled connection string), parameterised SQL in repositories. Supabase service-role client is NOT used for data access.	Real transactions (create submission + version + audit atomically), per-request org context, simple parameterised queries, same SQL runs in tests.
Tenant isolation	App-layer: every repo method takes `orgId` from `req.auth`. DB-layer: add `org_id` to every child table; RLS enabled with **no policies for anon/authenticated** (deny-all via the Data API); server connects with a privileged role. Optional extra: per-transaction `set_config('app.current_org', ...)`.	Closes the 'RLS decorative' gap honestly: public API keys can read nothing; the server enforces scoping and tests prove it.
IDs	All primary keys are real **uuid**. Seed data uses fixed, documented UUID constants. Social tables move to uuid too.	Matches schema.sql; removes type mismatches.
Sessions	Access JWT (HS256, 15 min, includes `sid` and `tv` token_version) + opaque refresh token (random 256-bit, stored only as SHA-256 hash in `auth_sessions`, rotated on every use, httpOnly+Secure+SameSite=Strict cookie).	Short-lived access, revocable sessions, instant invalidation on deactivation/role change/password change.
Schema source of truth	**Only** `db/migrations/*.sql`, applied by a real runner. `db/schema.sql` becomes a generated snapshot or is deleted.	One truth; reproducible databases.

PART 3 - Phase 0: baseline and safety

- Create branch `round3/secure-db`. Run `npm install`, `npm run lint`, `npm test`, `npm run build`, `npm run contract:check`. Record exact results in `docs/ROUND3_BASELINE.md`.
- Write `docs/ROUND3_AUDIT.md` re-verifying the 10 audit findings above against the code you see (confirm or correct each, with file and line). If a finding is wrong, say so.
- Add dependencies: `pg`, `@types/pg`, `cookie-parser`, `@types/cookie-parser`. Do not add an ORM.
- Create `.env.example` with placeholders only (see Appendix B) and confirm `.env`, `.env.*` (except example) are in `.gitignore`.

Checkpoint 0
Show the baseline command output and the audit confirmation table. Stop for approval before changing code.

PART 4 - Phase 1: configuration and secrets hardening

- Create `server/config.ts` that parses env with zod once at boot and exports a frozen `config` object: NODE_ENV, PORT, APP_URL, DATABASE_URL, JWT_SECRET (min 32 chars, reject known defaults and the old fallback string), JWT_ACCESS_TTL (default 15m), REFRESH_TTL_DAYS (default 7), COOKIE_SECURE, GEMINI_API_KEY (optional), GEMINI_MODEL, BCRYPT_COST (default 12), MAX_FAILED_LOGINS (5), LOCKOUT_MINUTES (15).
- Remove `JWT_SECRET` fallback in `server/auth.ts`. In test mode (VITEST) read a dedicated `JWT_SECRET` from `vitest.config.ts` env, not a hard-coded production default.
- Delete `src/services/supabase.ts` (or move it to `server/` without any hard-coded URL or key and without browser exposure). Remove every import of it from `src/`, including `src/api/client.ts`, `SettingsPage.tsx` and the `SupabaseHealthReport` type usage (replace with a plain type from the API contract).
- Replace Express `cors({ origin: process.env.APP_URL || true })` with an explicit allow-list from config; credentials enabled only for that origin. Add `express.json({ limit })` sensible limits (e.g. 2 MB, with a higher limit only on the upload routes) and a global error handler that never returns stack traces.
- Make `/api/readiness` check a real `SELECT 1` through the pool and return 503 if the database is unreachable. `/api/health` must not expose internal details such as key presence or hosts.

Acceptance
Server refuses to start with missing/weak JWT_SECRET or DATABASE_URL in every mode except tests. A production-build grep for 'supabase.co', 'sb_publishable', 'service_role' and 'inkloom_gemini_key' in `dist/` returns nothing.

PART 5 - Phase 2: schema reconciliation and a real migration runner

5.1 Migration runner (replace db/migrate.ts)

- Connect with pg using DATABASE_URL. Create `schema_migrations(filename text primary key, checksum text not null, applied_at timestamptz default now())`.
- For each `db/migrations/NNN_*.sql` in order: skip if already applied (verify checksum equals; if the file changed after being applied, fail loudly), otherwise run inside one transaction, insert the row, commit. On any error: roll back, print the SQL error and file, exit code 1.
- Add `npm run db:migrate:status` and `npm run db:reset:test` (drops and recreates a dedicated **test schema** only, refuses to run if DATABASE_URL points at a non-test database or NODE_ENV=production).

5.2 Fix the schema (new migrations; do not edit applied ones)

Create `006_reconcile.sql` onward. First inspect `001_init.sql` and `schema.sql`, pick 001 as base, and resolve differences in new migrations. Required changes:

- **org_id on every tenant child table** (brand_guidelines, approved_facts, terminology, brand_assets, content_submissions, content_versions, analysis_reports, detected_issues, correction_suggestions, review_comments, approval_records, social_accounts, publish_jobs): add `org_id uuid NOT NULL REFERENCES organisations`, backfill from the parent, and add a composite index `(org_id, ...)`. Use composite FKs where practical so a child can never point at a parent in another org.
- **Social tables to uuid:** `social_accounts.account_id` and `publish_jobs.job_id` are currently `text`. Convert (or recreate; this feature is simulated) and fix the variant references.
- **auth_sessions** table: `session_id uuid pk, user_id, org_id, refresh_token_hash text unique, created_at, last_used_at, expires_at, revoked_at, user_agent, ip_hash, replaced_by uuid`.
- **users additions:** `token_version int not null default 0`, `locked_until timestamptz`, `must_change_password boolean default false`, `password_changed_at timestamptz`, `last_login_at timestamptz`.
- **Optimistic concurrency:** add `row_version int not null default 1` and `updated_at` (with trigger) on brand_profiles, brand_guidelines, approved_facts, terminology, content_submissions.
- **Version numbering race:** ensure `UNIQUE (submission_id, version_no)` on content_versions. Allocate the next number inside a transaction (`SELECT ... FOR UPDATE` on the submission row).
- **Suggestion state:** make correction_suggestions the authoritative store for accepted/edited/dismissed status, final_text and dismiss_reason (today it lives inside in-memory report objects). Add `decided_by` and `decided_at`.
- **Immutability triggers:** keep append-only on content_versions, audit_logs, approval_records. Add a trigger preventing change of `org_id` on any tenant row. Add a trigger or constraint so approval_records.version_id must belong to the same org.
- **Indexes:** email lookup is on the citext column; add `UNIQUE(lower(email))` if login is global by email, or change login to require org/domain (see 7.1).

5.3 Fixed seed IDs and a truthful seed script

- Replace string IDs in `db/seed.ts` with deterministic UUID constants (e.g. `00000000-0000-4000-8000-0000000000a1` pattern, documented in the file). Keep all five README personas and **add the missing viewer** (Kavita Iyer, viewer@lumora.test) so the role is testable.
- Seed through the same pool inside one transaction, idempotent (`INSERT ... ON CONFLICT DO NOTHING/UPDATE`). Seed guidelines, facts, terms, brand profile, palette, **and** the 25 historical submissions currently generated in `repos.ts` (flag `is_demo = true`). Delete that in-memory generator from the repos.
- Seed passwords: bcrypt-hash at runtime from `SEED_PASSWORD` env (default allowed only when NODE_ENV is not production). Refuse to seed demo users in production unless `ALLOW_DEMO_SEED=true`.
- Seed must print row counts per table and exit non-zero on any error.

> **Checkpoint 2**
> Show `npm run db:migrate` twice (second run applies nothing), `db:migrate:status`, `db:seed` output with row counts, and a `\dt` / information_schema listing proving all tables exist with org_id columns.

PART 6 - Phase 3: Row Level Security as defence in depth

- New migration: `ALTER TABLE ... ENABLE RLS` and `FORCE ROW LEVEL SECURITY` on every tenant table. Revoke all table privileges on these tables from `anon` and `authenticated` roles. Create **no** permissive policies for those roles (default deny).

- Add a org-scoped policy set for the server role using `current_setting('app.current_org', true)::uuid` (note the correct cast; empty setting must yield no rows, not an error). In repositories, run each request inside a transaction that does `SELECT set_config('app.current_org', $1, true)` so RLS is genuinely exercised. If the server role has BYPASSRLS, document it and make the application-level scoping the tested guarantee.
- Remove the four old policies from 003 that used the wrong pattern (via a new migration that drops and recreates them).
- Add a test that connects with the Supabase **anon key over the REST API** (if you have the URL/key in the test env) and confirms a `select` on every tenant table returns zero rows or 401/403. If anon credentials are not available in the test env, skip with an explicit message in the report (never silently pass).

PART 7 - Phase 4: persistent, org-scoped repositories

Rewrite `server/db/repos.ts` (split into `server/db/*.repo.ts` files if it helps) so every exported repo keeps its current method names but becomes async SQL via the pg pool. Remove all module-level data arrays.

7.1 Repository rules

- Every method that touches tenant data takes `orgId` as a required parameter and includes `WHERE org_id = $n`. Methods that previously took only an ID (`suggestionRepo.getById`, `reportRepo.getByVersionId`, `commentRepo`, `approvalRepo`) must now require `orgId`. This fixes audit finding 6.
- `userRepo.getById` may stay org-agnostic only for the auth middleware; it must return only the fields needed (never the password_hash outside the login path) and the middleware must verify `user.org_id === claims.org`.
- Login lookup: pick one and document it: (a) email is globally unique via `UNIQUE(email)`, or (b) login requires the organisation domain/slug. Existing README flow uses email only, so default to (a) and enforce it in a migration.
- Use parameterised queries only. No string-built SQL. If you need dynamic ORDER BY or filters, use a whitelist map.
- Return the same object shapes the routes and `src/types/inkloom.ts` expect (snake_case fields, `issues[]` nested in reports, suggestion embedded in each issue). Add a mapper layer where DB rows differ.
- Pagination: list endpoints (submissions, audit logs, review queue, comments) accept `limit` (max 100) and `cursor/offset`; default sorted by created_at DESC. Do not return unbounded lists.

7.2 Transactions (must be atomic)

Operation	Single transaction must contain
POST /api/submissions	insert submission + version 1 + audit log
POST /api/submissions/:id/analyse	insert analysis_report + detected_issues + correction_suggestions (replace previous report for that version) + audit log + status update
POST /api/submissions/:id/versions	lock submission row, compute next version_no, insert version, update status, audit log
POST /api/versions/:id/decision	re-check self-approval guard, insert approval_record, update submission status, audit log (all-or-nothing)
POST /api/auth/register-organisation	insert organisation + brand profile + first admin user + audit log
PATCH /api/users/:id (role/deactivate)	last-admin check with row lock, update user, bump token_version, revoke sessions, audit log

7.3 Move the AI pipeline fully server-side

- `POST /api/submissions/:id/analyse` already exists: ensure it loads guidelines, facts and terms from the database by org and profile, runs `runAiPipeline` with `config.GEMINI_API_KEY` only, and persists the report. Remove any code path where the browser supplies an API key or runs the pipeline.
- Respect `brand_profiles.external_ai_allowed`: when false, the server must run code-only checks and record `ai_status = 'external_ai_disabled'`.

Acceptance
`grep -n "^let " server/db/*.ts` shows no data arrays. A test starts the app, creates a submission through the API, re-imports/restarts the app module (new pool), and reads the same submission back.

PART 8 - Phase 5: secure sign-in and session handling

8.1 Endpoints (extend, keep existing paths)

Endpoint	Behaviour
POST /api/auth/login	Generic error on any failure. Track failed_login_count; after MAX_FAILED_LOGINS set locked_until (LOCKOUT_MINUTES) and return the same generic 401 (optionally 429 with Retry-After). Reset counter on success. Do a dummy bcrypt compare for unknown emails to equalise timing. On success: create auth_sessions row, return access token in body and refresh token in httpOnly cookie. Audit login_success / login_failed / account_locked.
POST /api/auth/refresh	Reads cookie, looks up SHA-256 hash, rejects if missing/expired/revoked. **Rotation:** issue a new refresh token, mark the old one replaced. If an already-replaced token is presented (reuse), revoke the entire session family and audit 'refresh_reuse_detected'.
POST /api/auth/logout	Revoke current session, clear cookie, audit logout.
POST /api/auth/logout-all	Revoke all sessions of the caller; bump token_version.
POST /api/auth/change-password	Requires current password; enforce policy (min 10 chars, not equal to email/name, not in a small blocklist such as 'Password123!'); bump token_version; revoke other sessions; audit.
GET /api/auth/me	Return user + onboarding flags computed from DB (replaces the client's store-based getAuthMe).
POST /api/users (admin)	Creates user with a one-time temporary password (returned once, never stored in plain text) and `must_change_password = true`. Reject role values outside the 4 roles. Email uniqueness error must not leak other orgs.

8.2 Middleware changes

- `requireAuth`: verify HS256 only; require `sid` exists and session not revoked/expired; require `claims.tv === user.token_version`; reject deactivated or locked users; reject when `must_change_password` except for change-password, me and logout. Cache the user/session lookup for a few seconds at most (or not at all) so deactivation takes effect immediately.
- Set cookies with `httpOnly: true, sameSite: 'strict', secure: config.COOKIE_SECURE, path: '/api/auth'`. Because the refresh cookie is SameSite=Strict and scoped to /api/auth, state-changing routes use Bearer tokens only and need no CSRF token; document this reasoning in SECURITY.md.
- Rate limiting: keep the login limiter; add per-IP and per-user limiters on /api/auth/refresh, /api/users, uploads and /analyse (analysis is paid LLM usage). Key analysis limiter by user id, not only IP.
- `register-organisation`: keep, but add stricter rate limit, password policy, and a config flag `ALLOW_SELF_SERVE_ORG_SIGNUP` (default false in production). Use uuid ids, one transaction, and make the new user the first admin.

Acceptance
Tests prove: wrong password x5 locks the account; forged/expired/tampered tokens fail; logout makes the old access token unusable at next request is NOT required (15 min), but the refresh token must be dead; deactivating a user or changing their role invalidates their access token on the very next request (token_version); refresh-token reuse revokes the family.

PART 9 - Phase 6: complete backend role-based access control

Replace scattered `requireRole(...)` strings with one source of truth: `server/permissions.ts` exporting a typed permission map and `requirePermission(perm)`. Implement exactly the matrix below (it is the proposal's matrix, corrected where the current code is looser). Anything not listed is denied.

Permission	Admin	Creator	Reviewer	Viewer
auth.me / change password / logout	Y	Y	Y	Y
users.manage (list/create/update/deactivate)	Y	-	-	-
brand.read (profile, guidelines, facts, terms, assets)	Y	Y	Y	Y
brand.write (profiles, guidelines, facts, terms, assets, import)	Y	-	-	-
submission.create / edit / new version / send-for-review	Y	Y	-	-
submission.read - own	Y	Y	Y	Y*
submission.read - all in org	Y	if team_visibility=team	Y	approved only
analysis.run (costs LLM quota)	Y	Y (own)	-	-
suggestion.decide (accept/edit/dismiss)	Y	Y (own submission)	-	-
review.queue	Y	-	Y	-
review.decide (approve / request changes)	Y**	-	Y**	-
comment.read	Y	Y	Y	Y
comment.write	Y	Y (own submissions)	Y	- (read-only)
analytics.read	Y	Y	Y	Y
audit.read	Y	-	-	Y (read-only, per current matrix)
eval.run	Y	-	Y	-
social.* (providers, accounts, publish, schedule)	Y	Y (approved own only)	-	-

* Viewer sees only content already approved. ** Never on a submission they authored (self-approval guard stays).

Required fixes found in the audit

- Add `requireAuth` and permission checks to `GET /api/social/accounts`, `GET /api/social/providers` and role checks to the POST social routes. Social publishing must only be allowed for **approved** versions (server-side check), and everything stays labelled as simulated.
- `PATCH /api/suggestions/:id`: load the suggestion joined to submission and org; 404 (not 403) if outside the org; require the caller to be the submission creator or admin; submission must be in a state that allows correction (draft / in_correction); write decided_by/decided_at; audit it.
- `POST /api/submissions/:id/comments`: block viewers; creators may comment only on their own submissions; reviewers on anything in the review states.
- Object-level checks everywhere IDs are accepted (`:id` for users, guidelines, facts, terms, versions, suggestions). Never trust an org_id from the request body or query string.
- Creator list/read visibility must respect `brand_profiles.team_visibility` ('own' vs 'team') on the server (currently decided in `api/client.ts`).

- Remove or properly implement `POST /api/db/sync` (currently a stub that returns success). Replace `GET /api/db/status` output with real pool/migration status for admins only.
- Mass-assignment protection: zod schemas with `.strict()` on every body so clients cannot set role, org_id, status, creator_id, is_demo or version numbers.

Acceptance
A table-driven test iterates every (route x role) pair against the matrix above plus 'no token' and 'other-org token' and asserts 401/403/404 exactly as specified.

PART 10 - Phase 7: migrate the React client off local storage

This is the largest phase. The client must become a thin, server-driven UI. Do it page by page and keep the app working after each page.

10.1 Rewrite src/api/client.ts

- Every one of the ~47 methods calls the server through one private `request()` helper that adds the Bearer token, parses `{ data }`/`{ error, message }`, and throws a typed `ApiError`. Remove every `catch (_) {}` that falls back to `store` data, and every reference to `store`.
- Access token held **in memory only** (module variable), not in sessionStorage/localStorage. On page load call `/api/auth/refresh` (cookie) to restore the session; show a loading state while doing so.
- On a 401, attempt exactly one refresh then retry the original request once (de-duplicate concurrent refreshes with a shared promise). If refresh fails, clear state and route to login with a 'session expired' message. Support multi-tab logout via the `storage` event or `BroadcastChannel`.
- Remove `getCurrentUser()`'s sessionStorage `inkloom_active_user` hack and any persona-switching that imitates users. The current user comes only from `/api/auth/me`. Quick-login buttons on the login page may remain but must go through real `/api/auth/login` and only render when `VITE_SHOW_DEMO_LOGINS=true`.

10.2 Retire src/services/store.ts as a data source

- Delete `loadFromStorage`, `persist`, `resetToSeed` and all seed arrays from the browser bundle. Move `LABELLED_TEST_SET` to the server/eval side (`db/eval/testset.json` already exists) without expanding it (Improvement 2 owns benchmarking).
- Update every file that imports `store`: AppShell, EvidencePopover, SocialPublishModal, AiEngineModal, ApprovalPage, SubmissionPage, AuditLogPage, SettingsPage, UsersPage, MySubmissionsPage, GuidelinesPage, BrandHubPage, SocialPublishingPage, DashboardPage. Replace with API calls and local React state (loading, error, empty states for each).
- Remove `localStorage.getItem/setItem('inkloom_gemini_key')` in AiEngineModal and SettingsPage. The AI engine panel becomes a read-only status display fed by the server ('external AI enabled/disabled for this profile', 'analysis last ran with: ...'); only an admin can toggle `external_ai_allowed` on the profile.
- **Draft autosave:** replace `localStorage['inkloom_draft_autosave']` with server-side drafts: create the submission with status `draft` on first save, then `PATCH` body text with a debounce (e.g. 2 s) and show 'Saved at hh:mm'. Large base64 media must be uploaded through the existing upload path or object storage, not stored in the DB row as a base64 blob; enforce size and MIME limits server-side.

10.3 UX for multi-user reality

- Handle 403 by hiding/disabling the control *and* showing a clear message if reached anyway; the UI role is cosmetic, the server is the guard.

- Show 'last updated by X at Y' on guidelines, facts, submissions. On a 409 `stale_version` response show a 'This item changed, reload to see the latest' banner rather than overwriting.
- Refetch on window focus and every 30-60 s on Review Queue, My Submissions and Dashboard so two users see each other's changes without a manual refresh. (Realtime channels are optional and out of scope.)
- Add loading skeletons and error banners; never display stale seed data when the API fails.

Acceptance
`grep -rn "localStorage\|sessionStorage" src/` returns nothing except an optional non-sensitive UI preference key. `grep -rn "services/store\|services/supabase" src/` returns nothing. Opening the app in two browsers as creator and reviewer shows the same data.

PART 11 - Phases 8-10: concurrency, audit, tests and documentation

Phase 8: concurrency and integrity

- Optimistic locking: PATCH/PUT bodies include `row_version`; the update uses `WHERE ... AND row_version = $n` and returns `409 stale_version` when 0 rows change.
- Status transitions validated with `statusMachine.ts` inside the transaction with `SELECT ... FOR UPDATE` so two reviewers cannot both decide the same version (second gets 409 `already_decided`).
- Idempotency: `Idempotency-Key` header support on POST /api/submissions and /decision to survive double-clicks and retries.

Phase 9: audit and security headers

- Audit log every security-relevant event: login success/failure, lockout, refresh reuse, logout(-all), password change, user create/role change/deactivate, brand-knowledge writes, analysis run, suggestion decisions, review decisions, permission denials (403) on write routes. Store actor, org, entity, action, metadata, salted ip hash (salt from config, not a constant), user agent.
- Audit and approval reads: `GET /api/audit-logs` paginated and filterable by action/actor/date; admin full, viewer read-only per matrix.
- Helmet CSP tuned for the Vite build (no inline eval), HSTS in production, `app.set('trust proxy', ...)` configured correctly behind Render/Railway so rate limiting sees the real IP.

Phase 10: automated tests (must run against real Postgres)

Use a dedicated test database/schema. `vitest.config.ts` sets env; a global setup runs migrations + seed once; each suite truncates or uses transactions. Do not mock the database layer. Keep the existing 23 tests passing (adapt them to async repos) and add:

Suite	Must prove
persistence.test.ts	Create user, submission, version, comment, approval; recreate the app/pool; all rows still present with correct values.
tenant-isolation.test.ts	Two orgs. Org A token cannot read, list, patch or delete any org B resource across every :id route (expects 404). Includes the suggestion IDOR regression.
rbac-matrix.test.ts	Table-driven route x role x (no token / wrong org) matrix from Part 9.
session.test.ts	Lockout, refresh rotation, refresh reuse revokes family, logout revokes, token_version invalidates after role change/deactivation, expired token, wrong algorithm (alg=none) rejected.
governance-guards.test.ts	Self-approval, unresolved high findings, last-admin, immutable audit/approvals/versions (direct SQL UPDATE/DELETE must raise).
concurrency.test.ts	Parallel version creation yields distinct version_no; double decision gives one approval and one 409; stale row_version gives 409.
migrations.test.ts	Fresh DB migrates cleanly; second run is a no-op; checksum tampering fails.
rls.test.ts	Anon/authenticated roles read nothing; org GUC isolates rows when set.
bundle-secrets.test.ts	After `vite build`, no supabase URL, anon/service keys, JWT secret or Gemini key strings in `dist/`.

Phase 11 (documentation, honest)

- Rewrite `docs/ARCHITECTURE.md`, `SECURITY.md`, `DEVIATIONS.md`, `README.md` (env table with placeholders, no real project URL/keys), `API_CONTRACT.md` (new auth endpoints, pagination, 409 codes) and `DEMO_SCRIPT.md` to describe what is now **actually** true. Remove claims that were not true before (e.g.

'database single source of truth' while using in-memory arrays) and note the change in a 'What changed in Round 3' section.

- Add `docs/DEPLOYMENT.md`: Supabase setup, DATABASE_URL (pooled string), running migrations in CI/deploy, env vars on Render/Railway, HTTPS and cookie settings, backup/restore note, and how to rotate JWT_SECRET (invalidate sessions) and the DB password.

PART 12 - Verification, demo script and final report

12.1 Commands that must pass

```
npm run lint
npm run db:migrate && npm run db:migrate          # 2nd run = no-op
npm run db:seed                                   # prints per-table row counts
npm test                                          # real Postgres, all suites green
npm run build && npm run contract:check
grep -rn "localStorage\|sessionStorage" src/     # expect: nothing sensitive
grep -rn "supabase.co\|sb_publishable\|service_role" dist/ src/   # expect: nothing
grep -n "^let " server/db/*.ts                    # expect: no data arrays
```

12.2 Live demo script (for judges)

1. Terminal A: start the server. Browser 1: log in as **creator** (Ishaan), submit a draft, run analysis, accept one fix, send for review.

2. Browser 2 (private window): log in as **reviewer** (Neil). The submission appears in the Review Queue within one refresh interval. Add a comment, request changes.

3. Browser 1: the comment and status appear. Create version 2.

4. Stop and restart the server. Reload both browsers: sessions restore via refresh cookie; all submissions, versions, comments and decisions are still there.

5. Log in as **viewer** (Kavita): she can read approved content and analytics; every write button is disabled and a direct API write returns 403.

6. As **admin** (Maya): deactivate the creator; the creator's next request fails immediately and the Audit Log shows the event. Show the 'Reviewer cannot approve own submission' 403.

7. Show a second organisation's data is invisible (test output or a second registered org).

12.3 Final report format the agent must return

Section	Content
Summary	What was completed per phase (0-11) with commit/diff overview.
Evidence	Pasted output of every command in 12.1, test counts per suite, migration status.
Audit table	The 10 original findings with status: Fixed / Partially fixed / Not fixed (reason).
Deviations	Anything that differs from this prompt, with reasoning and risk.
Known limitations	Honest list (e.g. realtime not implemented, anon-key RLS test skipped if no credentials).
Manual steps for the team	Secrets to set, key rotation, Supabase settings, deployment steps.

Never claim
'Fully secure', '100% tests passing' without output, or 'persisted' unless a restart test proves it. The proposal promises honest reporting; keep that standard.

Appendix A - Suggested SQL sketches (adapt, do not paste blindly)

```
-- auth_sessions
CREATE TABLE auth_sessions (
  session_id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid NOT NULL REFERENCES users ON DELETE CASCADE,
  org_id uuid NOT NULL REFERENCES organisations ON DELETE RESTRICT,
  refresh_token_hash text NOT NULL UNIQUE,
  family_id uuid NOT NULL,
  replaced_by uuid,
  created_at timestamptz NOT NULL DEFAULT now(),
  last_used_at timestamptz,
  expires_at timestamptz NOT NULL,
  revoked_at timestamptz,
  user_agent text,
  ip_hash text
);
CREATE INDEX idx_sessions_user ON auth_sessions(user_id) WHERE revoked_at IS NULL;

-- users hardening
ALTER TABLE users
  ADD COLUMN token_version int NOT NULL DEFAULT 0,
  ADD COLUMN locked_until timestamptz,
  ADD COLUMN must_change_password boolean NOT NULL DEFAULT false,
  ADD COLUMN password_changed_at timestamptz,
  ADD COLUMN last_login_at timestamptz;

-- default-deny RLS for public API roles
ALTER TABLE content_submissions ENABLE ROW LEVEL SECURITY;
ALTER TABLE content_submissions FORCE ROW LEVEL SECURITY;
REVOKE ALL ON content_submissions FROM anon, authenticated;
CREATE POLICY org_scope ON content_submissions
  USING (org_id = NULLIF(current_setting('app.current_org', true), '')::uuid);

-- version numbering safety
ALTER TABLE content_versions
  ADD CONSTRAINT uq_version_no UNIQUE (submission_id, version_no);
```

Appendix B - Environment variables (.env.example, placeholders only)

Variable	Required	Notes
NODE_ENV	Yes	development | test | production
PORT, APP_URL	Yes (prod)	APP_URL is the single allowed CORS origin
DATABASE_URL	Yes	Supabase pooled Postgres connection string; server only
JWT_SECRET	Yes	>= 32 random chars; no fallback; rotate to invalidate all sessions
JWT_ACCESS_TTL / REFRESH_TTL_DAYS	No	15m / 7
COOKIE_SECURE	Yes (prod)	true behind HTTPS
MAX_FAILED_LOGINS / LOCKOUT_MINUTES	No	5 / 15
BCRYPT_COST	No	12
IP_HASH_SALT	Yes	Random salt for audit IP hashes
GEMINI_API_KEY / GEMINI_MODEL	Optional	Server only; absent = code-only checks
ALLOW_SELF_SERVE_ORG_SIGNUP	No	false in production

Variable	Required	Notes
ALLOW_DEMO_SEED / SEED_PASSWORD	Dev only	Never enabled in production
TEST_DATABASE_URL	For tests	Separate DB/schema; reset script refuses anything else
VITE_API_BASE / VITE_SHOW_DEMO_LOGINS	No	Only non-secret client flags use the VITE_ prefix

Appendix C - Out of scope (do not touch in this task)

- Benchmark expansion, precision/recall dashboards and regression harness (Round 3, Improvement 2).
- Automated image/logo/colour analysis (Round 3, Improvement 3).
- Real social-network publishing (keep simulated and labelled).
- Realtime websockets, SSO/OAuth, multi-factor auth, email verification (list as future scope in docs if desired).
- UI redesign. Only change UI as needed for server-driven data, loading/error states and role-aware controls.

One-line prompt to start
"Read the attached INKLOOM AI Round 3 Improvement 1 master prompt and my project. Start with Phase 0 only, show me the baseline results and your audit confirmation, then wait for my approval before Phase 1."